

CENTRO UNIVERSITÁRIO SENAC
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PROJETO INTEGRADOR - DESENVOLVIMENTO PARA DISPOSITIVOS
MÓVEIS:
ASTROMACHINE

Guilherme da Silva Serafim: 1143043623
Gustavo Magalhães Prada de Castro: 1142536492

São Paulo – 2026

Sumário

1. Introdução.....	3
1.1. Visão do Produto (Product Vision)	3
1.2. Escopo do Projeto	3
2. Desenvolvimento do Projeto	4
2.1 MVP – Produto Mínimo Viável.....	4
2.2. Estruturação do Planejamento Ágil	4
2.3 Pesquisa de Mercado e Design de Interface (Wireframing).....	5
2.4 História do Usuário	7
2.5 Definição de Pronto	8
2.6 Roadmap do Projeto.....	8
2.7 Stakeholders	8
2.8 Configuração do Ambiente e Interface Inicial.....	9
2.9 Modelagem e Estrutura do Banco de Dados.....	11
2.10 Monitoramento e Sprint Review	15
2.11 Análise e Mitigação de Riscos.....	18
3. Conclusão.....	20
4. Bibliografia.....	20

1. Introdução

1.1. Visão do Produto (Product Vision)

1.1.1 Nome do Projeto

AstroMachine

1.1.2 Problema a Ser Resolvido

Entusiastas de hardware e astronomia não encontram no mercado opções de computadores que unam performance técnica a uma estética personalizada (pintura, gravura e iluminação) baseada no espaço sideral.

1.1.3 Visão do Produto

Uma aplicação mobile que oferece serviços de montagem e personalização de PCs com temas de galáxias e exoplanetas, facilitando a escolha e contratação de builds exclusivas.

1.1.4 Público-Alvo

- Usuários principais: Gamers e entusiastas de hardware.
- Usuários secundários: Admiradores de astronomia e design.

1.2. Escopo do Projeto

1.2.1 Escopo do Produto

- Cadastro e login de usuários.
- Tela principal com catálogo de PCs já personalizados (ex: Build Andromeda, PC Orion).
- Módulo Administrativo (CRUD): Inserir, Alterar, Excluir e Consultar os modelos de customização no catálogo.
- Carrinho de compras para os serviços selecionados.
- Formulário para simulação de pagamento.
- Interface Acessível: Implementação de padrões básicos de acessibilidade (contraste, suporte a leitores de tela e fontes escaláveis) para garantir a inclusão de todos os perfis de entusiastas.

1.2.2 Fora do Escopo

- Relatórios avançados.
- Integração com gateway de pagamentos.

2. Desenvolvimento do Projeto

2.1 MVP – Produto Mínimo Viável

2.1.1 Objetivo do MVP

Validar a jornada de contratação de serviços de customização de hardware temático, garantindo que o entusiasta consiga selecionar uma "build" espacial e que o administrador consiga gerenciar esse catálogo de forma centralizada e ágil.

2.1.2 Funcionalidades Essenciais do MVP

Funcionalidade	Descrição	Prioridade
Vitrine Espacial	Exibição dos modelos de PCs personalizados.	Alta
Painel de Controle	Interface de CRUD para o administrador gerenciar os serviços (Inserir, Alterar, Excluir, Consultar).	Alta
Fluxo de Reserva	Adição ao carrinho e formulário de checkout para fechamento do pedido.	Alta
Autenticação e Perfil	Cadastro e login para personalização de hardware e acesso seguro ao sistema.	Alta
Acessibilidade Digital	Suporte a leitores de tela e padrões de contraste WCAG em todas as telas.	Média

2.2. Estruturação do Planejamento Ágil

2.2.1. Processo SCRUM

Para o desenvolvimento do AstroMachine, será adotado o framework Scrum adaptado para uma equipe de dois desenvolvedores, com Sprints de periodicidade semanal.

Papéis (Roles): A equipe dividirá as responsabilidades para garantir a agilidade no desenvolvimento:

- Guilherme Serafim: Atuará como Product Owner (PO), responsável pela definição de requisitos e prioridades do produto, acumulando também a função de Developer em React Native.
- Gustavo Magalhães: Atuará como Scrum Master (SM), focando na gestão do processo ágil e na remoção de impedimentos, acumulando também a função de Developer em React Native.

Artefatos: O projeto será guiado pelo Product Backlog (lista de todas as histórias de usuário) e pelo Sprint Backlog (conjunto de tarefas selecionadas para a semana), gerenciados de forma compartilhada pela dupla.

Cerimônias: Serão realizadas reuniões de Sprint Planning no início de cada ciclo para definição de metas, e sessões de Sprint Review/Retrospective ao final de cada semana para validação do código produzido e melhoria contínua dos processos da equipe.

2.2.2. Estimativa de Esforço (Planning Poker)

A complexidade técnica de cada funcionalidade foi estimada individualmente através da técnica de Planning Poker, utilizando a sequência de Fibonacci (1, 2, 3, 5, 8) para atribuir pontos de esforço.

ID	Funcionalidade / História	Prioridade	Esforço (Pontos)
US01	Cadastro e login de usuários	Alta	3
US02	Catálogo de PCs personalizados (Vitrine)	Alta	5
US03	Módulo Administrativo (CRUD completo)	Alta	8
US04	Carrinho de Compras e Pagamento	Média	5
US05	Acessibilidade e Inclusão	Média	3

2.3 Pesquisa de Mercado e Design de Interface (Wireframing)

2.3.1 Pesquisa Exploratória de Mercado

Concorrentes Diretos: Empresas como Pichau e Terabyte Shop (Brasil), que focam em performance gamer genérica, e boutiques como Origin PC (Global), focadas em estética premium, mas sem nicho na New Space Economy.

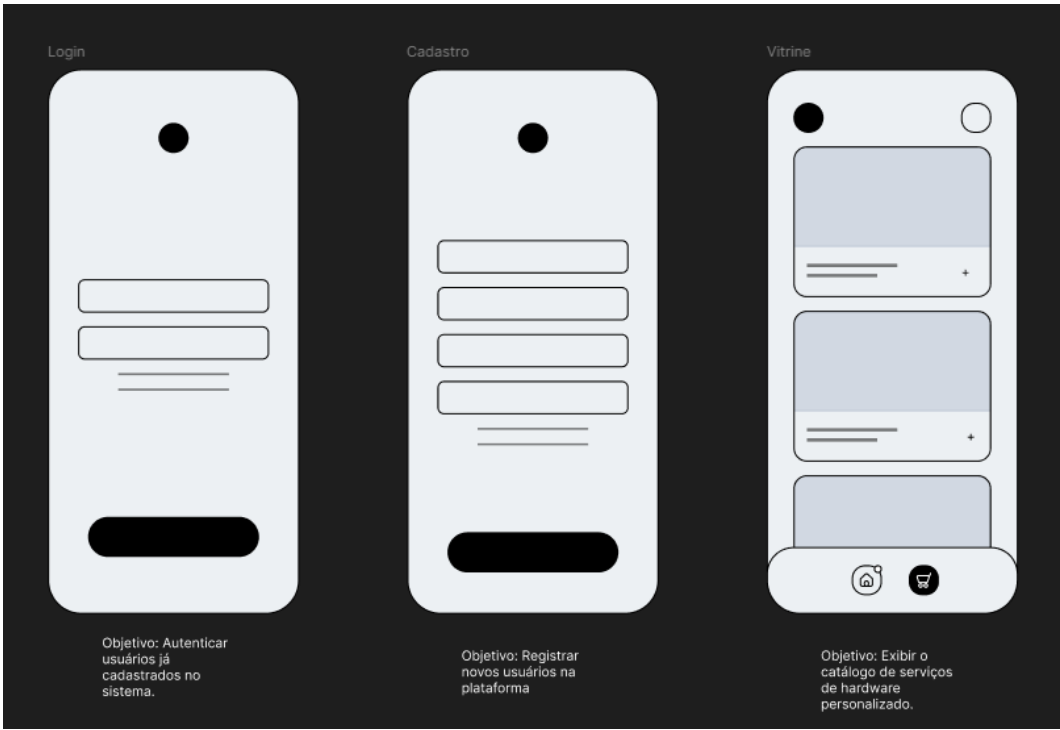
Diferencial AstroMachine: Especialização estética e técnica em hardware com temática espacial (nebulosas e exoplanetas). Este MVP foca exclusivamente na venda e personalização de builds, validando a marca como uma base sólida ("alicerce") para a futura expansão de serviços técnicos especializados no setor aeroespacial.

2.3.2 Wireframes

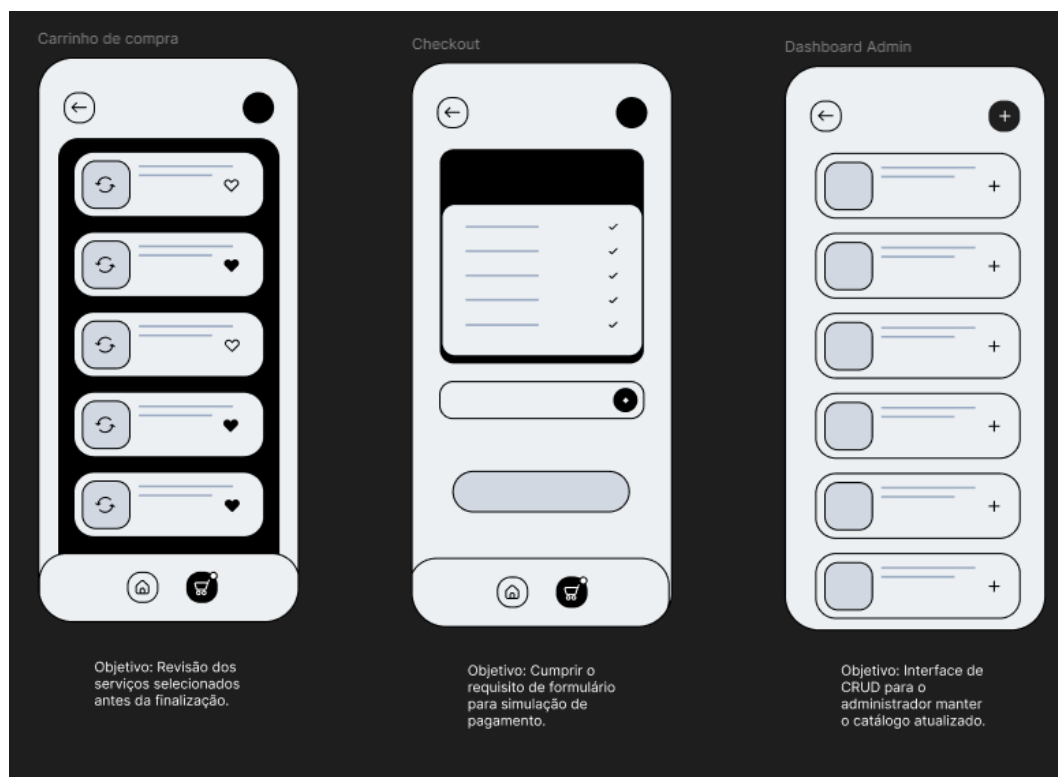
Os wireframes a seguir representam as seis telas do MVP, conforme projetado na Fase 1 do projeto. Cada tela foi desenhada priorizando a clareza do fluxo do usuário e a consistência visual com a temática espacial.

Tela	Objetivo
Login	Autenticar usuários já cadastrados no sistema.
Cadastro	Registrar novos usuários na plataforma.
Vitrine (Tela Principal)	Exibir o catálogo de serviços de hardware personalizado.
Carrinho de Compra	Revisão dos serviços selecionados antes da finalização.
Checkout	Cumprir o requisito de formulário para simulação de pagamento.
Dashboard Admin	Interface de CRUD para o administrador manter o catálogo atualizado.

Wireframes – Telas de Login, Cadastro e Vitrine



Wireframes – Telas de Carrinho, Checkout e Dashboard Admin



2.4 História do Usuário

ID	História do Usuário	Prioridade
US01	Como cliente, quero me cadastrar e realizar login, para salvar minhas preferências de customização.	Alta
US02	Como cliente, quero visualizar o catálogo de PCs personalizados, para escolher uma build baseada no tema espacial.	Alta
US03	Como administrador, quero inserir, alterar e excluir produtos do catálogo (CRUD), para manter a vitrine de hardware atualizada.	Alta
US04	Como cliente, quero adicionar itens ao carrinho e preencher o formulário de pagamento, para finalizar a contratação do	Média

	serviço de montagem.	
US05	Como usuário com baixa visão, quero poder ajustar o contraste e o tamanho das fontes no app, para navegar pelo catálogo com autonomia.	Média

2.5 Definição de Pronto

- Funcionalidade totalmente implementada em React Native conforme o protótipo.
- Código versionado com sucesso no repositório GitHub do grupo.
- Interface mobile operando sem erros fatais ou travamentos aparentes.
- Código devidamente comentado para facilitar a manutenção futura.
- História validada e aprovada pelo professor orientador da disciplina.
- Interface acessível validada pelas APIs de acessibilidade do React Native.

2.6 Roadmap do Projeto

2.6.1 Planejamento por Sprints

Sprint	Objetivo	Principais Entregas
Sprint 1	Planejamento Inicial	Documento de Visão, Backlog Priorizado e Wireframes.
Sprint 2	Estrutura e Gestão	Telas de Login e o Módulo Administrativo (CRUD de produtos).
Sprint 3	Fluxo do Cliente	Tela Principal (Catálogo), Carrinho e Fluxo de Pagamento.
Sprint 4	Finalização	Refinamento de UI/UX, gravação do vídeo demo e entrega final.

2.7 Stakeholders

- Público-alvo: Startups da nova economia espacial e entusiastas de hardware especializado.
- Público com necessidades específicas: Usuários que dependem de tecnologias assistivas e interfaces acessíveis.
- Orientador: Wilson da Silva Lourenço.
- Desenvolvedores Responsáveis: Guilherme da Silva Serafim e Gustavo Magalhães Prada de Castro.

2.8 Configuração do Ambiente e Interface Inicial

Esta seção documenta a configuração do ambiente de desenvolvimento e a criação do projeto base em React Native com Expo, garantindo que o aplicativo compile e inicie corretamente em emuladores Android e iOS.

2.8.1 Ferramentas e Versões Utilizadas

Ferramenta	Versão / Configuração
VS Code	1.99.x com extensões: React Native Tools, ESLint, Prettier
Node.js	22.x LTS
Expo CLI / SDK	SDK 54 (npx create-expo-app)
React Native	0.81.5
TypeScript	5.9.x
Zustand (State)	5.x
Android Studio / Emulador	Android 14 (API 34)
Git / GitHub	Repositório privado do grupo

2.8.2 Criação do Projeto Base

O projeto foi criado utilizando o Expo CLI com template TypeScript. A estrutura de pastas adotada separa as telas (screens), serviços de comunicação com a API (services), gerenciamento de estado (store), tipagem (types) e configuração de navegação (navigation), conforme demonstrado a seguir:

```
AstroMachine/
├─ mobile/
|   └─ App.tsx
|   └─ src/
|       └─ screens/           # Login, Cadastro, Vitrine, Carrinho, Checkout,
AdminDashboard, AdminForm
|       └─ services/         # api.ts (Axios + Node backend)
|       └─ store/            # authStore.ts, cartStore.ts (Zustand)
|       └─ types/            # index.ts (interfaces do domínio)
|       └─ theme/            # index.ts (cores, espaçamentos, fontes)
|       └─ utils/            # validation.ts, format.ts, dialog.ts
|       └─ db/               # database.native.ts, database.web.ts (SQLite)
|       └─ navigation/      # AppNavigator.tsx
|       └─ package.json
├─ server/
|   └─ src/
|       └─ index.ts          # Entry point Express
```

```

|   |   | routes/           # auth, products, services, appointments, checkout
|   |   | middlewares/      # auth.ts (JWT)
|   |   | models/           # types.ts
|   |   | services/         # database.ts (in-memory DB)
|   |   └─ package.json
|   └─ README.md

```

2.8.3 Telas Implementadas na Sprint 2

As seguintes telas foram implementadas, compiladas e validadas sem erros fatais nesta sprint:

Tela	Descrição	Status
Login	Campos de e-mail e senha + botão de acesso	Concluída
Cadastro	Formulário com nome, e-mail e senha	Concluída
Admin Dashboard	Listagem com opções de Inserir, Editar e Excluir produto	Concluída
Admin Form	Formulário de criação/edição de produto com validação	Concluída
Vitrine (Catálogo)	Grid de cards com builds espaciais e botão "Adicionar ao Carrinho"	Concluída
Detalhe do Produto	Tela com especificações técnicas completas e ação de compra	Concluída
Carrinho de Compras	Lista de itens selecionados com quantidade e valor total	Concluída
Checkout	Formulário de simulação de pagamento (dados fictícios)	Concluída
Acessibilidade	Tela de configuração de contraste e tamanho de fonte	Concluída

2.9 Modelagem e Estrutura do Banco de Dados

Esta seção apresenta o Diagrama de Entidade-Relacionamento (DER), a arquitetura do backend em Node.js e a configuração do banco de dados local mobile em SQLite.

2.9.1 Arquitetura de Dados

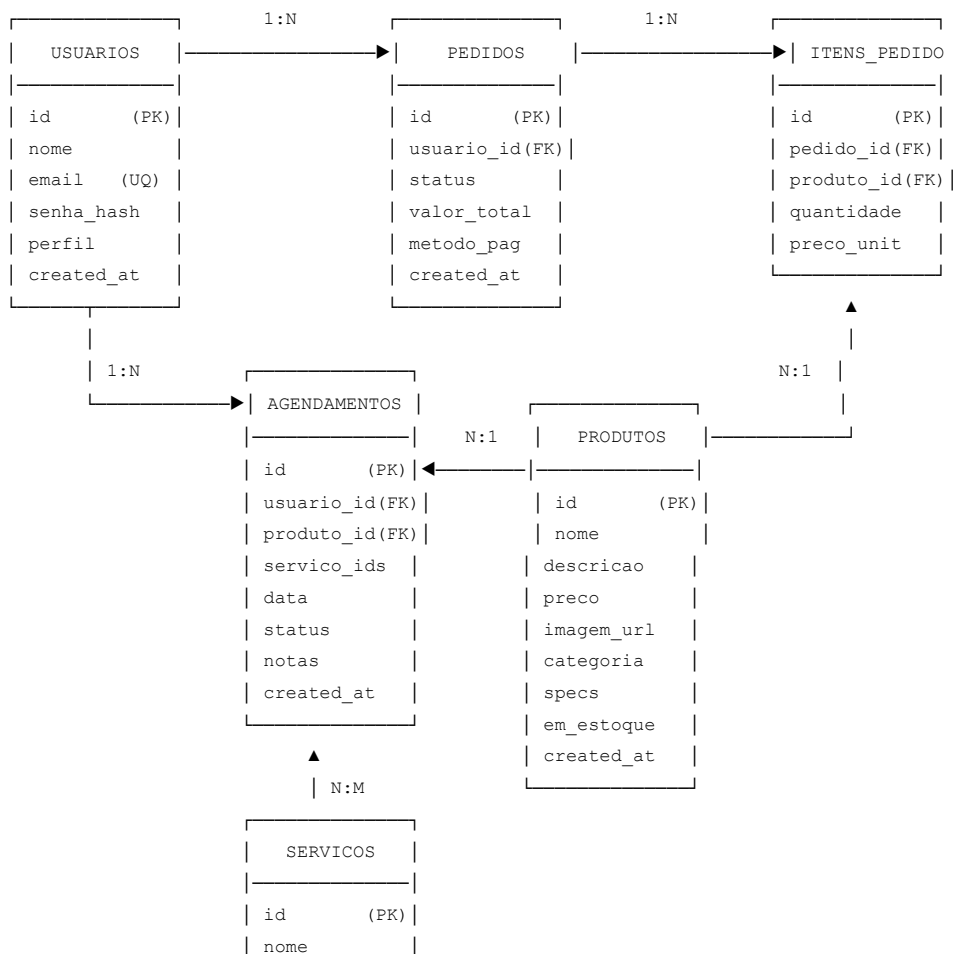
O projeto adota uma arquitetura de dois níveis de persistência:

- Backend (Servidor): API RESTful desenvolvida em Node.js + Express com TypeScript, utilizando banco de dados em memória (in-memory) que centraliza os dados de produtos (catálogo), serviços, agendamentos, usuários e pedidos.
- Mobile Local (Cache): O aplicativo utiliza expo-sqlite para armazenar localmente os dados do carrinho de compras e as informações do usuário autenticado via Zustand + AsyncStorage, garantindo funcionamento offline parcial.

2.9.2 Diagrama de Entidade-Relacionamento (DER)

O DER a seguir modela as seis entidades principais do sistema e seus relacionamentos:

Diagrama Entidade-Relacionamento – AstroMachine



descricao
preco
horas_est
categoria
created_at

Tabela: usuarios

Atributo	Tipo	Restrição
id	VARCHAR (UUID)	PK
nome	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
senha_hash	VARCHAR(255)	NOT NULL
perfil	ENUM('cliente','admin')	DEFAULT 'cliente'
created_at	DATETIME	DEFAULT NOW()

Tabela: produtos

Atributo	Tipo	Restrição
id	VARCHAR (UUID)	PK
nome	VARCHAR(100)	NOT NULL
descricao	TEXT	—
preco	DECIMAL(10,2)	NOT NULL
imagem_url	VARCHAR(255)	—
categoria	VARCHAR(80)	NOT NULL
specs	TEXT	—
em_estoque	BOOLEAN	DEFAULT TRUE
created_at	DATETIME	DEFAULT NOW()

Tabela: servicos

Atributo	Tipo	Restrição
id	VARCHAR (UUID)	PK
nome	VARCHAR(100)	NOT NULL
descricao	TEXT	—
preco	DECIMAL(10,2)	NOT NULL
horas_estimadas	INTEGER	DEFAULT 1
categoria	VARCHAR(80)	NOT NULL
created_at	DATETIME	DEFAULT NOW()

Tabela: agendamentos

Atributo	Tipo	Restrição
----------	------	-----------

id	VARCHAR (UUID)	PK
usuario_id	VARCHAR (UUID)	FK → usuarios.id
produto_id	VARCHAR (UUID)	FK → produtos.id (opcional)
servico_ids	JSON (array de UUIDs)	FK → servicos.id
data	DATETIME	NOT NULL
status	ENUM('pendente','confirmado','concluido','cancelado')	DEFAULT 'pendente'
notas	TEXT	—
created_at	DATETIME	DEFAULT NOW()

Tabela: pedidos

Atributo	Tipo	Restrição
id	VARCHAR (UUID)	PK
usuario_id	VARCHAR (UUID)	FK → usuarios.id
status	ENUM('pendente','confirmado','entregue')	DEFAULT 'pendente'
valor_total	DECIMAL(10,2)	NOT NULL
metodo_pagamento	VARCHAR(100)	NOT NULL
created_at	DATETIME	DEFAULT NOW()

Tabela: itens_pedido

Atributo	Tipo	Restrição
id	VARCHAR (UUID)	PK
pedido_id	VARCHAR (UUID)	FK → pedidos.id
produto_id	VARCHAR (UUID)	FK → produtos.id
nome_produto	VARCHAR(100)	NOT NULL
quantidade	INTEGER	NOT NULL
preco_unitario	DECIMAL(10,2)	NOT NULL

Relacionamentos:

- Um usuário pode ter vários pedidos (1:N) e vários agendamentos (1:N).
- Um pedido é composto por vários itens (1:N). Cada item referencia um produto do catálogo (N:1).
- Um agendamento referencia opcionalmente um produto (N:1) e pode incluir múltiplos serviços (N:M via array JSON).

2.9.3 Configuração do Backend Node.js

A API foi estruturada em Node.js com Express e TypeScript, com as seguintes rotas principais:

Método	Rota	Descrição	Autenticação
POST	/auth/register	Cadastro de novo usuário com hash bcrypt	Pública
POST	/auth/login	Autenticação com retorno de JWT	Pública
GET	/products	Listagem pública do catálogo	Pública
GET	/products/:id	Detalhes de um produto	Pública
POST	/products	Inserção de novo produto	Admin
PUT	/products/:id	Atualização de produto existente	Admin
DELETE	/products/:id	Exclusão de produto	Admin
GET	/services	Listagem de serviços	Pública
POST	/services	Inserção de novo serviço	Admin
PUT	/services/:id	Atualização de serviço	Admin
DELETE	/services/:id	Exclusão de serviço	Admin
GET	/appointments	Listagem de agendamentos (filtrado por perfil)	Autenticado
POST	/appointments	Criação de agendamento	Autenticado
POST	/checkout/simulate	Simulação de pedido com itens do carrinho	Autenticado

2.10 Monitoramento e Sprint Review

Esta seção registra a primeira Sprint Review realizada ao final da Sprint 2, conforme a cerimônia prevista no framework Scrum adotado pelo projeto.

2.10.1 Reunião de Sprint Review – Sprint 2

Item	Detalhes
Data da Reunião	01/04/2026
Participantes	Guilherme Serafim (PO), Gustavo Magalhães (SM/Dev), Prof. Wilson da Silva Lourenço (Orientador)
Duração	45 minutos (online via Google Meet)
Objetivo	Apresentar as interfaces desenvolvidas na Sprint 2 e validar o backlog para Sprint 3

2.10.2 Interfaces Apresentadas

Foram demonstradas ao orientador as telas de Login, Cadastro e Admin Dashboard, todas funcionando no emulador Android. A navegação entre as telas foi apresentada via navegação customizada implementada com gerenciamento de estado local (useState).

2.10.3 Feedback Recebido e Ajustes no Backlog

Feedback do Orientador	Ação Definida	Prioridade
Validação de formulários (e-mail e senha) ausente nas telas de autenticação	Implementar validação com Zod + React Hook Form na Sprint 3	Alta
Ausência de mensagem de feedback ao usuário após ações (ex: produto adicionado ao carrinho)	Adicionar diálogos de confirmação via Alert nativo	Média
Botão de logout não visível no Admin Dashboard	Adicionar botão de logout no header do painel	Alta

2.10.4 Relatório de Status – Sprint 2

Métrica	Planejado	Realizado	Observação
Story Points da Sprint	14	11	US03 (CRUD) parcialmente concluída

Telas Entregues	3	3	Login, Cadastro e Admin Dashboard
Bugs Críticos	0	1	Crash no Android ao navegar sem token JWT – corrigido
Cobertura de Acessibilidade	50%	30%	Labels de acessibilidade adicionados na Sprint 3

Velocidade da equipe: 11 pontos/sprint. Meta ajustada para Sprint 3: 13 pontos, considerando a conclusão do CRUD e o desenvolvimento do fluxo do cliente (Vitrine, Carrinho e Checkout).

2.10.5 Pauta da Sprint Review – Sprint 2

#	Item da Pauta	Responsável	Tempo
1	Abertura e objetivos da reunião	Gustavo (SM)	5 min
2	Demonstração das telas implementadas (Login, Cadastro, Admin Dashboard)	Guilherme (PO)	15 min
3	Apresentação da estrutura do backend Node.js e rotas da API	Guilherme (PO)	10 min
4	Feedback do orientador sobre as interfaces e a modelagem	Prof. Wilson	10 min
5	Revisão do backlog e definição de prioridades para Sprint 3	Todos	5 min

2.10.6 Ata da Sprint Review – Sprint 2

Item	Detalhes
Data	01/04/2026
Horário	19h00 – 19h45
Local	Google Meet (remoto)

Participantes	Guilherme Serafim, Gustavo Magalhães, Prof. Wilson da Silva Lourenço
Registrador	Gustavo Magalhães

Pontos discutidos:

- As telas de Login e Cadastro foram apresentadas com navegação funcional. O orientador aprovou o visual e sugeriu adicionar validação de campos.
- O Admin Dashboard foi demonstrado com listagem de produtos e operações de inserção e exclusão. A edição foi apresentada parcialmente implementada.
- O orientador destacou a necessidade de feedback visual ao usuário após ações (ex: toast/alert ao adicionar produto ao carrinho).
- Foi decidido priorizar a validação de formulários e o botão de logout para a Sprint 3.

Decisões tomadas:

- Implementar validação com Zod + React Hook Form em todas as telas de formulário.
- Adicionar botão de logout visível no header do Admin Dashboard.
- Manter o escopo do MVP sem redução, ajustando apenas a velocidade da sprint.

Próximos passos:

- Sprint 3: Concluir CRUD completo, implementar Vitrine, Carrinho e Checkout.
- Adicionar labels de acessibilidade em todas as telas.
- Preparar demonstração funcional para a próxima Sprint Review.

2.11 Análise e Mitigação de Riscos

Esta seção apresenta a identificação sistemática dos riscos do projeto AstroMachine e os respectivos planos de mitigação, com foco nos riscos de alta prioridade.

2.11.1 Identificação de Riscos

ID	Risco	Probabilidade	Impacto	Prioridade	Categoria
R01	Falha ou instabilidade da API Node.js em ambiente de produção/demonstração	Média	Alto	Alta	Técnico
R02	Incompatibilidade de bibliotecas React Native com versão do Expo SDK	Média	Alto	Alta	Técnico
R03	Atraso nas entregas por sobrecarga acadêmica dos desenvolvedores	Alta	Médio	Alta	Cronograma
R04	Redução da equipe de 3 para 2 integrantes (já ocorrida)	Ocorrido	Alto	Alta	Pessoas
R05	Erros de sincronização de dados entre SQLite local e backend remoto	Baixa	Médio	Média	Técnico
R06	Interface não atender padrões WCAG de acessibilidade	Média	Baixo	Baixa	Qualidade

2.11.2 Plano de Mitigação para Riscos de Alta Prioridade

R01 – Falha na API Node.js

Plano A: Implementar tratamento de erros (try/catch) em todas as chamadas à API com mensagens de feedback ao usuário. Plano B: Caso a API esteja indisponível durante a demonstração final, utilizar dados mockados (JSON estático) armazenados localmente no aplicativo, simulando o retorno da API sem dependência de rede.

R02 – Incompatibilidade de Bibliotecas

Plano A: Fixar versões de todas as dependências críticas no package.json (sem uso de "^" ou "~") para evitar atualizações automáticas. Plano B: Em caso de conflito irresolúvel, substituir a

biblioteca problemática por alternativa compatível (ex: substituir react-native-vector-icons por @expo/vector-icons, nativo do Expo).

R03 – Atraso por Sobrecarga Acadêmica

Plano A: Manter quadro Kanban atualizado semanalmente no GitHub Projects, com revisões toda segunda-feira. Plano B: Priorizar as funcionalidades de maior nota (autenticação e CRUD) e adiar funcionalidades de menor prioridade (acessibilidade avançada) caso o cronograma seja comprometido.

R04 – Redução da Equipe (Risco Ocorrido)

Ação Tomada: Com a saída de um integrante, as responsabilidades foram redistribuídas entre Guilherme (PO + Dev) e Gustavo (SM + Dev). O escopo do MVP foi mantido integralmente, porém o ritmo das sprints foi recalibrado de 3 integrantes para 2, reduzindo a velocidade esperada de 21 para 14 pontos/sprint. O roadmap original permanece exequível dentro do prazo semestral.

3. Conclusão

Ao término da Fase 2 do projeto AstroMachine, o grupo concluiu a configuração do ambiente de desenvolvimento, a implementação das primeiras telas funcionais em React Native com TypeScript e a modelagem completa do banco de dados, incluindo as seis entidades exigidas (Usuários, Produtos, Serviços, Agendamentos, Pedidos e Itens de Pedido). A primeira Sprint Review confirmou a viabilidade técnica do projeto e gerou ajustes importantes no backlog para as próximas sprints.

A adaptação da equipe, reduzida de três para dois integrantes, foi gerenciada de forma estruturada com redistribuição de papéis e ajuste de velocidade, sem comprometimento do escopo essencial do MVP. O plano de mitigação de riscos documentado oferece alternativas concretas para os principais pontos de falha identificados, garantindo a continuidade do desenvolvimento.

As fases seguintes concentrarão esforços na conclusão do fluxo completo do cliente (Vitrine, Carrinho e Checkout), no refinamento de UI/UX com a estética espacial e na preparação do vídeo demonstrativo para a entrega final.

4. Bibliografia

- SENAC. Roteiro do Projeto Integrador: Desenvolvimento para Dispositivos Móveis. São Paulo: Centro Universitário Senac, 2026.
- REACT NATIVE. Documentação oficial: Components and APIs. Disponível em: <https://reactnative.dev/>. Acesso em: 28 fev. 2026.
- SCHWABER, Ken; SUTHERLAND, Jeff. O Guia do Scrum. Scrum.org, 2020.
- EXPO. Documentação oficial: expo-sqlite. Disponível em: <https://docs.expo.dev/versions/latest/sdk/sqlite/>. Acesso em: 01 abr. 2026.
- EXPRESS. Documentação oficial: Express.js. Disponível em: <https://expressjs.com/>. Acesso em: 01 abr. 2026.
- ZUSTAND. Documentação oficial: Zustand State Management. Disponível em: <https://zustand-demo.pmnd.rs/>. Acesso em: 01 abr. 2026.
- ZOD. Documentação oficial: TypeScript-first schema validation. Disponível em: <https://zod.dev/>. Acesso em: 01 abr. 2026.